

Project Report: Text Similarity & Duplicate Detection

1. Project Objective

The primary goal of this project is to build a machine learning pipeline capable of classifying and flagging whether two pieces of text content are duplicates of one another. This task involves defining and executing a complete end-to-end pipeline: feature extraction, data preprocessing, model building, training, and rigorous evaluation. To demonstrate a clear progression in modeling sophistication, three distinct experimental approaches were implemented, moving from shallow lexical matching to deep semantic representations.

2. Dataset & Preprocessing

The experiments were conducted using the Quora Question Pairs Dataset.

- **Data Structure:** The dataset consists of hundreds of thousands of question pairs accompanied by binary ground-truth labels indicating whether the questions share the same core intent (duplicates).
- **Preprocessing:** Real-world textual data inherently contains missing or corrupted values. Rows containing null values (NaN) were scrubbed from the dataset prior to feature extraction to prevent data-type errors during tokenization and encoding.
- **Resource Management:** Because the full dataset exceeds 400,000 rows, memory constraints (RAM and VRAM) were a primary consideration. To facilitate the more computationally expensive Transformer models, batch-processing and strategic sampling were utilized to ensure stable pipeline execution without triggering memory allocation failures.

3. Experimental Methodology

Approach 1: TF-IDF + Logistic Regression (Baseline)

The first experiment established a baseline using traditional Natural Language Processing

(NLP) techniques.

- **Feature Extraction:** Text was encoded using Term Frequency-Inverse Document Frequency (TF-IDF), a lexical representation method.
- **Model Building:** The resulting TF-IDF representations were horizontally aggregated and fed into a shallow Logistic Regression classifier.
- **Limitations:** While computationally efficient, TF-IDF relies on exact keyword matching and traditionally struggles with synonyms (e.g., failing to recognize that "laptop" and "computer" hold similar semantic meaning despite being different words).

Approach 2: Pretrained Sentence Transformers (Bi-Encoder)

The second approach upgraded the feature extraction phase by introducing dense vector embeddings.

- **Feature Extraction:** We utilized the all-MiniLM-L6-v2 pretrained model from the sentence-transformers library. In this "Bi-Encoder" architecture, each question in the pair is encoded independently into a high-dimensional vector.
- **Aggregation & Classification:** To aggregate the pair for classification, the absolute difference between the two question embeddings was calculated. This difference vector—a highly effective feature for similarity tasks—was then passed into a Logistic Regression model to determine duplication.

Approach 3: Fine-Tuning BERT (Cross-Encoder)

The final and most sophisticated approach transitioned from feature extraction to end-to-end deep learning.

- **Architecture:** We utilized a HuggingFace bert-base-uncased model. Unlike Approach 2, which processes questions independently, this "Cross-Encoder" feeds both questions into the model simultaneously.
- **Mechanism:** The questions are separated by a [SEP] token. This allows BERT's attention mechanism to directly compare the complex, nuanced interactions between every word in Question 1 against every word in Question 2.
- **Training:** The model was fine-tuned using a dedicated classification head configured for binary output (num_labels=2) on a T4 GPU.

4. Results & Evaluation

The models were evaluated on unseen test data using Accuracy, Precision, Recall, and F1-Scores.

Metric	Majority Baseline	Approach 1 (TF-IDF)	Approach 2 (Bi-Encoder)	Approach 3 (Cross-Encoder)
Accuracy	63.08%	72.97%	79.17%	83.35%
Precision (Weighted)	-	0.76	0.79	-
Recall (Weighted)	-	0.73	0.79	-
F1-Score (Weighted)	-	0.73	0.79	-

Evaluation Analysis

- **Baseline Superiority:** Approach 1 successfully out-performed a "dumb" majority baseline (which always guesses 0) by a margin of roughly 10%, validating the predictive power of the pipeline.
- **Semantic Understanding:** Moving from Approach 1 to Approach 2 yielded a massive performance jump, pushing accuracy to over 79%. The F1-score for actual duplicates (Class 1) improved notably. This confirms the hypothesis that the Sentence Transformer successfully captured the meaning behind the words rather than relying strictly on lexical overlap.
- **Precision vs. Recall:** In Approach 2, the model demonstrated a strong capability for identifying true negatives (0.84 precision/recall for Class 0), indicating it is highly effective at ensuring non-duplicates are not accidentally flagged (minimizing False Positives).
- **The Cross-Encoder Advantage:** Approach 3 yielded the highest overall performance, achieving a final evaluation accuracy of **83.35%** with an evaluation loss of 0.41. This confirms that feeding both questions simultaneously into BERT allows the attention mechanism to capture deeply nuanced semantic differences that the independent Bi-Encoders (Approach 2) missed. Despite the longer training time, the Cross-Encoder is definitively the most capable architecture for accurately flagging duplicate content.

5. Discussion & Conclusion

This project successfully established a robust classification pipeline for text duplicate detection.

The progression of experiments clearly demonstrates the evolution of NLP capabilities:

1. **Lexical limitations:** The TF-IDF baseline (Approach 1) proved that while word-frequency models are effective, they are bottlenecked by their inability to understand context.
2. **The power of dense vectors:** Approach 2 proved that encoding sentences into mathematical space significantly boosts performance (~79% accuracy), allowing the model to capture semantic synonyms effectively.
3. **The Cross-Encoder advantage:** While Bi-Encoders are faster, independently encoding sentences causes the model to miss highly complex semantic differences. Implementing the BERT Cross-Encoder (Approach 3) solves this by forcing the model's attention mechanism to interact with both strings simultaneously.

In conclusion, for tasks requiring scalable, rapid similarity searches across millions of documents, the Bi-Encoder (Approach 2) offers the best balance of speed and performance. However, for precise, highly nuanced duplicate flagging (such as moderating forums or filtering patents), the Cross-Encoder architecture (Approach 3) provides the necessary depth of contextual understanding.